

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

PRAKTIKUM 2

RELASI ANTAR OBJEK (CLASS DIAGRAM & ASOSIASI)



OLEH

Nama : Lexi Mulia Yunaspi

NIM : 2511531006

Kelas : B

MATA KULIAH PEMOGRAMAN BERORIENTASI OBJEK

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

ASISTEN PRAKTIKUM : EZZA ADDINI

Program Studi Informatika

Fakultas Teknologi Informasi

Universitas Andalas

2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan Praktikum Pemrograman Berorientasi Objek Praktikum 2 ini dapat diselesaikan. Laporan ini disusun sebagai dokumentasi kegiatan praktikum sekaligus sebagai bentuk pertanggungjawaban atas program yang telah dibuat dan diuji.

Pada praktikum kedua ini, program perbankan dari praktikum sebelumnya dikembangkan dengan menerapkan relasi antar objek. Program tidak lagi hanya menyimpan data rekening, tetapi juga dapat menyimpan objek Transaksi sebagai riwayat transaksi. Materi yang dipelajari meliputi pembuatan class, constructor, asosiasi antar class, ArrayList, pencatatan transaksi, serta fitur cetak mutasi rekening.

Laporan ini menjelaskan proses pengerjaan mulai dari persiapan project, latihan pembuatan class Transaksi dan relasinya dengan Rekening, sampai Tugas 1 dan Tugas 2. Penjelasan kode dibuat secara bertahap dan per bagian agar fungsi setiap baris atau kelompok baris kode dapat dipahami dengan lebih mudah.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran sangat diharapkan untuk perbaikan laporan dan pemahaman praktikum berikutnya. Semoga laporan ini dapat memberikan manfaat dan membantu dalam memahami penerapan konsep Pemrograman Berorientasi Objek menggunakan Java.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Tujuan	5
1.3 Manfaat	5
BAB II PEMBAHASAN.....	6
A. LATIHAN	9
2.1 Persiapan Project dan Package.....	9
2.2 Class Transaksi	10
2.3 Relasi Class Rekening dan Transaksi	11
2.4 Pencatatan Setor Tunai.....	13
B. TUGAS	14
2.5 Tugas 1 – Integrasi Rekam Jejak Penarikan	14
2.6 Tugas 2 – Fitur Cetak Mutasi Rekening	16
2.7 Penambahan Menu 6 pada Main	17
2.8 Integrasi Program pada Class Main	18
C. HASIL PENGUJIAN	23
2.9 Hasil Pengujian Praktikum.....	23
2.10 Rangkuman Latihan dan Tugas.....	29
BAB III PENUTUP.....	30
3.1 Kesimpulan	30
3.2 Saran.....	30
DAFTAR PUSTAKA.....	31

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek merupakan pendekatan pemrograman yang menggunakan class dan object sebagai dasar penyusunan program. Dengan konsep ini, data dan method yang berhubungan dapat ditempatkan dalam satu class sehingga program lebih terstruktur dan lebih mudah dikembangkan. Praktikum 2, program perbankan yang telah dibuat sebelumnya dikembangkan agar dapat menyimpan riwayat transaksi. Untuk memenuhi kebutuhan tersebut dibuat class Transaksi yang berhubungan dengan class Rekening. Satu rekening dapat mempunyai banyak transaksi sehingga hubungan tersebut dapat diterapkan menggunakan ArrayList. Pengembangan ini juga menghasilkan fitur baru, yaitu pencatatan transaksi setor dan tarik tunai serta fitur Cetak Mutasi. Dengan fitur tersebut, pengguna dapat melihat transaksi yang pernah dilakukan pada rekening aktif. Program ini menjadi contoh sederhana penerapan relasi antar objek dalam Java.

1.2 Tujuan

Tujuan praktikum ini adalah memahami penerapan relasi antar objek pada Java, membuat class Transaksi, menghubungkan class Transaksi dengan class Rekening, dan menyimpan banyak transaksi menggunakan ArrayList. Selain itu, praktikum bertujuan menerapkan pencatatan transaksi yang berhasil serta membuat fitur cetak mutasi rekening melalui menu program.

1.3 Manfaat

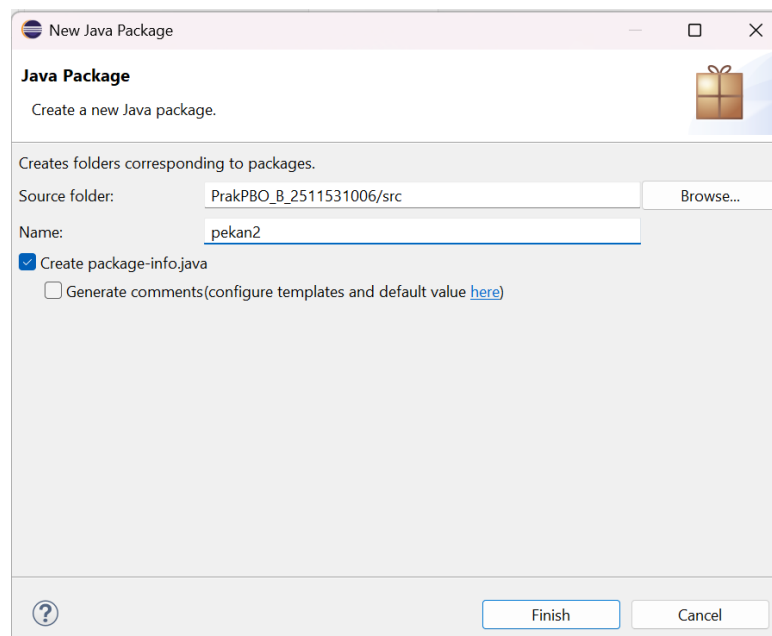
Praktikum ini membantu mahasiswa memahami bagaimana beberapa class dapat saling berhubungan dalam sebuah program. Mahasiswa juga dapat memahami penggunaan constructor, object, ArrayList, percabangan, perulangan for-each, dan pemanggilan method

antar objek. Hasil praktikum dapat menjadi dasar untuk membuat program PBO yang lebih kompleks.

BAB II

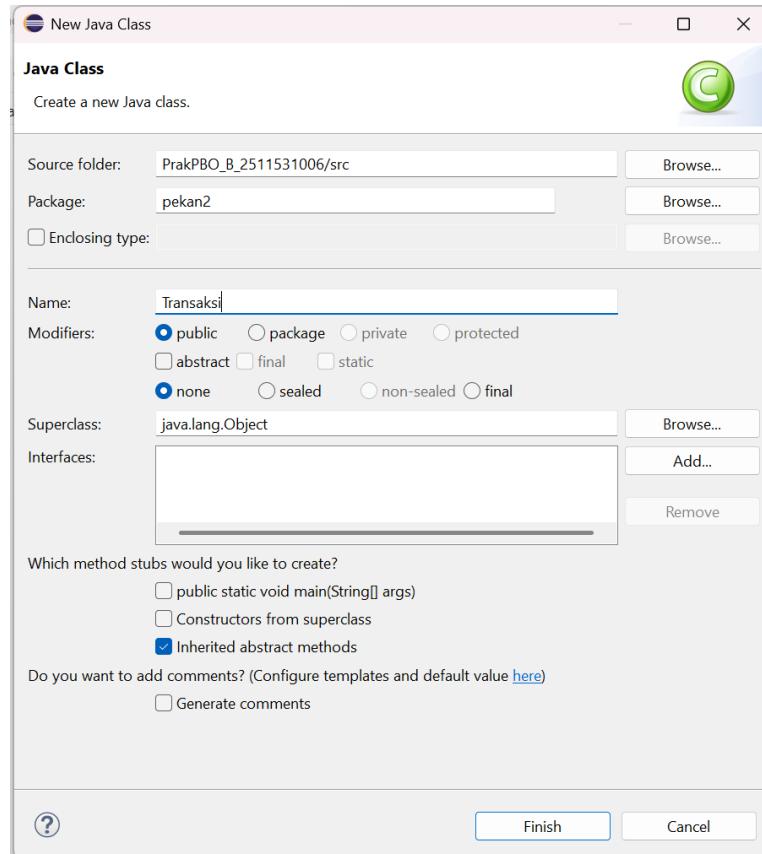
PEMBAHASAN

Gambar dibawah menunjukkan proses pembuatan package baru pada Eclipse. Source folder diarahkan ke project PrakPBO_B_2511531006/src dan nama package diisi pekan2. Package ini digunakan untuk menempatkan seluruh class Praktikum 2 agar terpisah dari package praktikum sebelumnya.



Gambar 2.1 Pembuatan package pekan2

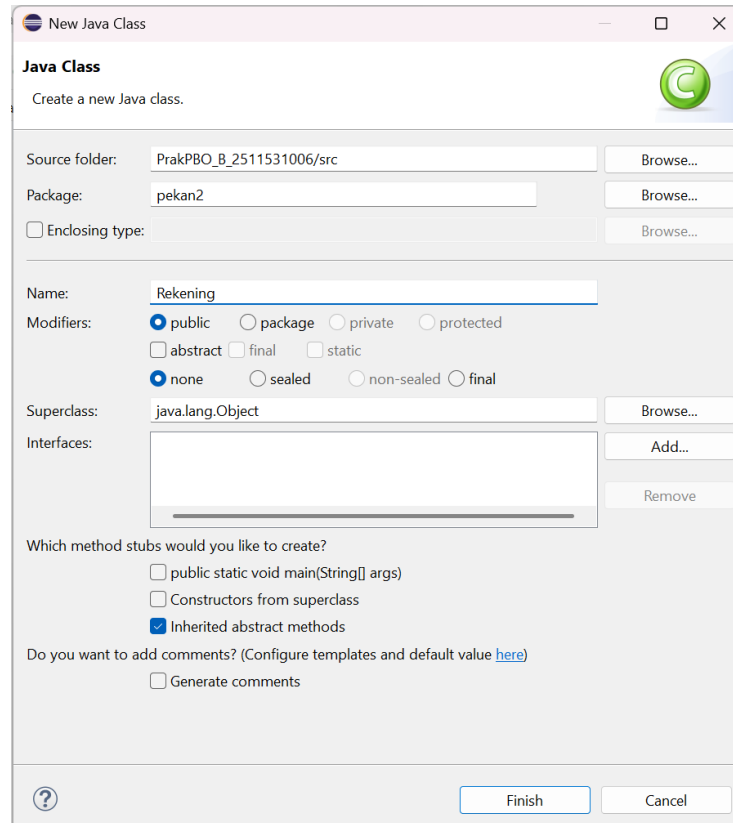
Gambar menunjukkan proses pembuatan class Transaksi di dalam package pekan2. Nama class diisi Transaksi dan modifier public dipilih agar class dapat digunakan oleh class lain. Method main tidak dipilih karena class ini berfungsi sebagai model data transaksi, bukan class utama.



Gambar 2.2 Pembuatan class Transaksi

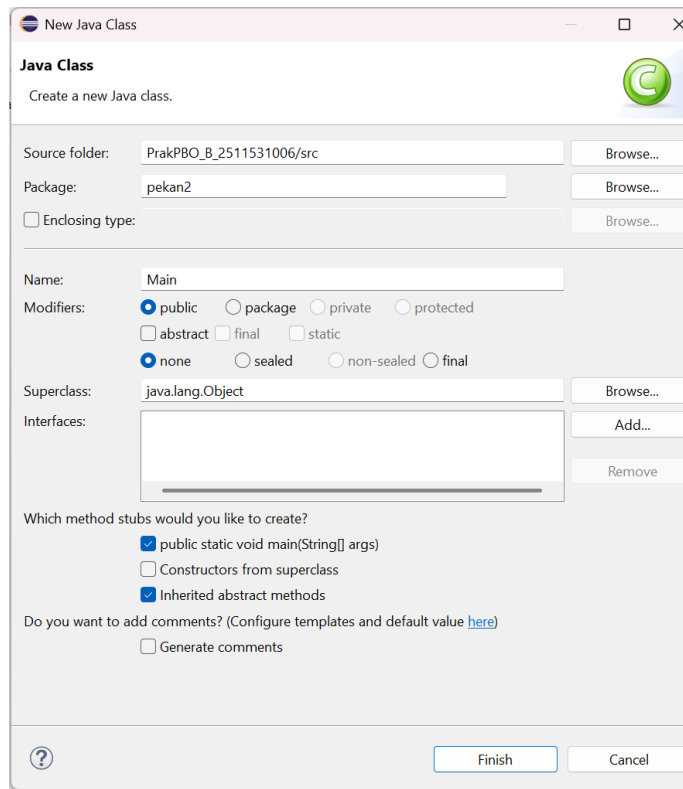
Gambar menunjukkan proses pembuatan class Rekening pada package pekan2. Nama class diisi Rekening dengan modifier public. Class ini nantinya digunakan untuk menyimpan nomor rekening, nama pemilik, saldo, serta kumpulan riwayat transaksi.

Gambar 2.3 Pembuatan class Rekening



Gambar 2.3 Pembuatan class Rekening

Gambar menunjukkan proses pembuatan class Main pada package pekan2. Opsi public static void main(String[] args) dipilih karena Main menjadi class utama untuk menjalankan program. Menu dan proses interaksi pengguna nantinya ditempatkan pada method main.



Gambar 2.4 Pembuatan class Main

A. LATIHAN

2.1 Persiapan Project dan Package

Praktikum 2 merupakan lanjutan dari program pada praktikum sebelumnya sehingga project yang digunakan tetap PrakPBO_B_2511531006. Pada source folder dibuat package baru bernama pekan2. Package tersebut digunakan untuk menyimpan class Transaksi, Rekening, dan Main pada Praktikum 2.

a. Package pada class

```
1 package pekan2;  
2
```

Baris package digunakan untuk menentukan tempat class dalam struktur project. Dengan menggunakan package pekan2, seluruh class Praktikum 2 dapat dikelompokkan dengan rapi dan tidak bercampur dengan class pada praktikum sebelumnya. Package juga membantu Java membedakan class yang mempunyai nama sama tetapi berada pada package berbeda. Pada package pekan2 terdapat tiga class utama. Transaksi bertugas menyimpan detail transaksi, Rekening menyimpan data rekening sekaligus riwayat transaksi, sedangkan Main mengatur input pengguna dan menu program. Pembagian tersebut membuat setiap class mempunyai fungsi yang lebih jelas.

2.2 Class Transaksi

Class Transaksi merupakan class baru pada Praktikum 2. Class ini digunakan untuk mewakili satu transaksi yang terjadi pada rekening. Data yang disimpan terdiri dari ID transaksi, jenis transaksi, dan nominal. Object Transaksi nantinya dimasukkan ke dalam ArrayList riwayatTransaksi milik class Rekening.

a. Package dan deklarasi class

```
1 package pekan2;  
2  
3 public class Transaksi {  
4
```

Baris package menempatkan class Transaksi pada package pekan2. Setelah itu public class Transaksi mendeklarasikan class yang dapat digunakan oleh class lain. Tanda kurung kurawal membuka isi class sehingga atribut, constructor, dan method dapat didefinisikan di dalamnya.

b. Atribut transaksi

```
4  
5     String idTransaksi;  
6     String jenis;  
7     double nominal;
```

Ketiga baris tersebut merupakan atribut yang menyimpan state dari object Transaksi. `idTransaksi` digunakan sebagai identitas transaksi, `jenis` menyimpan keterangan Kredit atau Debit, sedangkan `nominal` menyimpan jumlah uang yang terlibat. Tipe `double` dipakai untuk menyimpan nilai nominal.

c. Constructor Transaksi

```
8
9 public Transaksi(String idTransaksi, String jenis, double nominal) {
10     this.idTransaksi = idTransaksi;
11     this.jenis = jenis;
12     this.nominal = nominal;
13 }
```

Constructor digunakan untuk memberikan nilai awal ketika object Transaksi dibuat menggunakan keyword `new`. Parameter constructor menerima ID, jenis, dan nominal dari program yang membuat object. Keyword `this` digunakan untuk menunjukkan atribut milik object, sehingga parameter dan atribut yang mempunyai nama sama tetap dapat dibedakan.

d. Method `cetakDetail()`

```
13 }
14
15 public void cetakDetail() {
16     System.out.println("ID Transaksi : " + idTransaksi + " | Jenis : " + jenis + " | Nominal : Rp" + nominal);
17 }
18 }
```

Method `cetakDetail()` digunakan untuk menampilkan isi sebuah object Transaksi ke Console. `System.out.println` menggabungkan teks dengan nilai atribut menggunakan operator `+`. Ketika method ini dipanggil, ID, jenis, dan nominal transaksi akan ditampilkan dalam satu baris agar mudah dibaca.

2.3 Relasi Class Rekening dan Transaksi

Relasi antara Rekening dan Transaksi dibuat karena satu rekening dapat memiliki banyak transaksi. Hubungan tersebut diterapkan dengan membuat atribut `ArrayList<Transaksi>` pada class Rekening. Dengan demikian setiap object Rekening mempunyai tempat sendiri untuk menyimpan kumpulan transaksi yang dilakukan.

a. Import `ArrayList`

```
2  
3 import java.util.ArrayList;
```

Baris ini mengimpor class ArrayList dari library Java. ArrayList digunakan karena jumlah transaksi dalam satu rekening dapat bertambah selama program berjalan. Dengan ArrayList, object Transaksi dapat disimpan dan ditambahkan satu per satu.

b. Deklarasi class dan atribut rekening

```
4  
5 public class Rekening {  
6     String nomorRekening;  
7     String namaPemilik;  
8     double saldo;  
9 }
```

Baris pertama mendeklarasikan class Rekening. Tiga atribut berikutnya menyimpan nomor rekening, nama pemilik, dan saldo. Ketiga data tersebut merupakan state utama yang dimiliki oleh setiap object Rekening.

c. Membuat riwayat transaksi

```
10  
11 ArrayList<Transaksi> riwayatTransaksi = new ArrayList<Transaksi>();
```

Baris ini membuat ArrayList yang hanya menerima object bertipe Transaksi. Variabel riwayatTransaksi menjadi tempat penyimpanan seluruh transaksi milik rekening. Inilah bagian utama yang menunjukkan relasi antara object Rekening dengan banyak object Transaksi.

d. Constructor Rekening

```
12 public Rekening(String nomor, String nama, double saldoAwal) {  
13     nomorRekening = nomor;  
14     namaPemilik = nama;  
15     saldo = saldoAwal;  
16 }
```

Constructor Rekening menerima nomor, nama, dan saldo awal. Nilai tersebut kemudian disimpan ke atribut object agar setiap rekening mempunyai data awal yang berbeda. Constructor dijalankan saat object Rekening dibuat dengan keyword new.

2.4 Pencatatan Setor Tunai

Pada latihan ini method `setorTunai` dikembangkan agar transaksi yang berhasil tidak hanya mengubah saldo, tetapi juga dicatat sebagai object Transaksi. Jenis transaksi setor diberi nilai Kredit dan ID transaksi menggunakan awalan TRX-S-. Jika nominal tidak memenuhi syarat, program hanya menampilkan pesan gagal.

a. Method dan validasi

```
19  
20 public void setorTunai(double nominal) {
```

Method `setorTunai` menerima satu parameter nominal. Kondisi `if` memeriksa apakah nominal lebih besar dari nol. Jika kondisi benar, proses setoran dilanjutkan; jika tidak, program masuk ke bagian `else` dan transaksi tidak dicatat.

b. Menambah saldo

```
    saldo += nominal;
```

Operator `+=` menambahkan nilai nominal ke saldo yang sudah ada. Baris ini mengubah state saldo pada object Rekening. Perubahan saldo dilakukan terlebih dahulu sebelum transaksi berhasil dibuat dan disimpan.

c. Membuat transaksi Kredit

```
Transaksi transaksiBaru = new Transaksi("TRX-S-" + (riwayatTransaksi.size() + 1), "Kredit", nominal);  
riwayatTransaksi.add(transaksiBaru);  
System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
```

Setelah saldo berhasil ditambah, program membuat object Transaksi baru. ID diberi awalan TRX-S-, jenis transaksi diisi Kredit, dan nominal menggunakan nilai setoran. Object tersebut mewakili satu transaksi setor yang berhasil.

d. Menyimpan transaksi

```
    riwayatTransaksi.add(transaksiBaru);
```

Method add memasukkan object transaksiBaru ke dalam ArrayList. Setelah baris ini dijalankan, transaksi setor sudah menjadi bagian dari riwayat rekening dan dapat ditampilkan kembali melalui fitur cetak mutasi.

e. Kondisi gagal

```
} else {  
    System.out.println("Gagal: Nominal setor harus lebih dari 0!");  
}
```

Bagian else dijalankan jika nominal tidak lebih besar dari nol. Program menampilkan pesan bahwa nominal setor harus lebih dari 0. Karena tidak ada object Transaksi yang dibuat pada bagian ini, setoran gagal tidak masuk ke riwayat.

B. TUGAS

2.5 Tugas 1 – Integrasi Rekam Jejak Penarikan

Tugas 1 meminta agar setiap penarikan yang berhasil mempunyai rekam jejak transaksi. Program juga harus memastikan transaksi gagal tidak dicatat. Oleh karena itu, object Transaksi Debit dibuat hanya setelah pemeriksaan saldo dan batas minimum penarikan berhasil dilewati.

a. Method tarikTunai dan pemeriksaan saldo

```
34 public void tarikTunai(double nominal) {  
35  
36     if (nominal > saldo) {  
37  
38         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp"+ saldo);  
39     }
```

Method tarikTunai menerima nominal penarikan dari pengguna. Kondisi pertama memeriksa apakah nominal lebih besar daripada saldo yang tersedia. Jika benar, transaksi dihentikan dan hanya pesan kegagalan yang ditampilkan sehingga saldo dan riwayat tidak berubah.

b. Pemeriksaan minimum penarikan

```

40     } else if (nominal < 10000) {
41
42         System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");

```

Jika saldo mencukupi, program memeriksa syarat minimum penarikan. Nominal kurang dari Rp10.000 dianggap tidak memenuhi aturan program. Pada kondisi ini saldo tidak dikurangi dan transaksi juga tidak dimasukkan ke riwayat.

c. Penarikan berhasil

```

43
44     } else {
45
46         saldo -= nominal;

```

Blok else hanya dijalankan jika saldo mencukupi dan nominal tidak kurang dari Rp10.000. Operator -= mengurangi saldo sesuai jumlah uang yang ditarik. Setelah saldo berubah, program melanjutkan pencatatan transaksi.

d. Membuat transaksi Debit

```

    Transaksi transaksiBaru = new Transaksi("TRX-T-" + (riwayatTransaksi.size() + 1), "Debit", nominal);

```

Object Transaksi dibuat untuk merekam penarikan yang berhasil. ID menggunakan awalan TRX-T-, jenis transaksi diisi Debit, dan nominal berisi jumlah penarikan. Posisi kode ini penting karena object hanya dibuat setelah penarikan dinyatakan berhasil.

e. Menyimpan dan menampilkan transaksi

```

        riwayatTransaksi.add(transaksiBaru);
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    }
}

```

Object transaksiBaru ditambahkan ke riwayat sehingga penarikan berhasil dapat dilihat kembali. Setelah itu program menampilkan pesan keberhasilan dan saldo terbaru. Penarikan yang gagal tidak mencapai bagian add sehingga tidak dicatat sebagai transaksi berhasil.

2.6 Tugas 2 – Fitur Cetak Mutasi Rekening

Tugas 2 menambahkan method cetakMutasi pada class Rekening. Method ini bertugas membaca seluruh object Transaksi yang tersimpan dalam ArrayList dan menampilkannya. Jika belum ada transaksi, program menampilkan pesan khusus sesuai ketentuan tugas.

a. Method cetakMutasi

```
62 public void cetakMutasi() {  
63  
64     System.out.println("==MUTASI REKENING==");  
65 }
```

Method cetakMutasi dibuat pada class Rekening karena riwayat transaksi merupakan bagian dari data rekening. Perintah println pertama menampilkan judul agar hasil mutasi mudah dibedakan dari output menu atau transaksi lainnya.

b. Memeriksa riwayat kosong

```
66 if (riwayatTransaksi.isEmpty()) {  
67  
68     System.out.println("Belum ada transaksi pada rekening ini");  
69 }
```

Method isEmpty() digunakan untuk mengetahui apakah ArrayList tidak memiliki data. Jika kosong, program menampilkan pesan bahwa belum ada transaksi. Kondisi ini mencegah program melakukan perulangan terhadap data yang memang belum tersedia.

c. Menampilkan seluruh transaksi

```
69  
70 } else {  
71  
72     for (Transaksi transaksi : riwayatTransaksi) {  
73  
74         transaksi.cetakDetail();  
75     }  
76 }
```

Jika riwayat tidak kosong, program menggunakan for-each untuk mengambil setiap object Transaksi. Variabel transaksi mewakili satu object yang sedang diproses. Method cetakDetail kemudian dipanggil sehingga setiap transaksi ditampilkan satu per satu.

d. Penutup output


```
//
78         System.out.println("-----");
79     }
80 }
```

Garis pembatas digunakan untuk membuat tampilan Console lebih rapi. Kurung kurawal menutup method cetakMutasi setelah seluruh proses pemeriksaan dan pencetakan selesai.

2.7 Penambahan Menu 6 pada Main

Agar fitur cetak mutasi dapat digunakan pengguna, menu utama pada class Main ditambah pilihan nomor 6. Ketika pengguna memilih nomor tersebut, program akan memanggil cetakMutasi pada akun yang sedang aktif.

a. Menambahkan menu

```
--
24         System.out.println("5. Ganti Akun");
25         System.out.println("6. Cetak Mutasi (Riwayat)");
26         System.out.println("7. Keluar");
```

Ketiga baris tersebut menampilkan pilihan menu pada Console. Menu nomor 6 merupakan fitur baru untuk melihat riwayat transaksi, sedangkan nomor 0 tetap digunakan untuk mengakhiri program.

b. Case 6

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
```

case 6 dijalankan ketika pilihan pengguna bernilai 6. Sebelum mencetak mutasi, program memeriksa apakah akunAktif masih null. Jika belum ada rekening aktif, program menampilkan pesan kesalahan.

c. Memanggil cetakMutasi

```
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

Jika akun aktif tersedia, program memanggil method cetakMutasi pada object akunAktif. Dengan demikian data riwayat yang dicetak adalah milik rekening yang sedang dipilih. break menghentikan case 6 dan mengembalikan program ke menu utama.

2.8 Integrasi Program pada Class Main

Class Main menjadi penghubung seluruh class dalam program. Main menerima input pengguna, membuat rekening, memanggil method setor dan tarik, mengganti akun, serta menjalankan fitur cetak mutasi. Bagian ini menunjukkan bagaimana object Rekening dan Transaksi digunakan bersama dalam satu alur program.

a. Package dan import

```
1 package pekan2;  
2  
3 import java.util.ArrayList;  
4 import java.util.Scanner;
```

Package menentukan lokasi class Main. ArrayList diimpor untuk menyimpan daftar rekening, sedangkan Scanner digunakan untuk menerima input dari keyboard. Kedua library tersebut diperlukan agar menu dan penyimpanan beberapa rekening dapat berjalan.

b. Deklarasi class dan main

```
public class Main {  
    public static void main(String[] args) {
```

Baris pertama membuat class Main. Method main merupakan titik awal ketika program Java dijalankan. Parameter String[] args disediakan oleh Java untuk menerima argumen ketika program dijalankan dari lingkungan tertentu.

c. Membuat Scanner

```
Scanner input = new Scanner(System.in);
```

Object Scanner bernama input dibuat untuk membaca masukan pengguna. System.in menunjukkan bahwa sumber input berasal dari keyboard melalui

Console. Object ini digunakan pada proses input nomor rekening, nama, saldo, nominal setor, dan nominal tarik.

d. Daftar rekening dan akun aktif

```
ArrayList<Rekening> daftarRekening =new ArrayList<Rekening>();  
Rekening akunAktif = null;  
boolean isRunning = true;
```

daftarRekening menyimpan seluruh object Rekening yang dibuat pengguna. akunAktif menyimpan rekening yang sedang digunakan, sedangkan nilai null berarti belum ada akun aktif. Variabel isRunning digunakan sebagai pengendali perulangan menu.

e. Perulangan menu

```
14  
15 while (isRunning) {
```

While menjalankan menu selama isRunning bernilai true. Selama pengguna belum memilih menu keluar, program akan kembali menampilkan pilihan dan menerima input baru. Ketika isRunning diubah menjadi false, perulangan berhenti.

f. Membaca pilihan menu

```
26  
27     int pilihan = input.nextInt();  
28     input.nextLine();  
29  
30     switch (pilihan) {
```

nextInt membaca angka menu dan menyimpannya pada variabel pilihan. nextLine dipanggil setelahnya untuk membersihkan sisa enter sebelum program menerima input teks. Switch kemudian memilih proses berdasarkan nilai pilihan.

g. Case 1 membuka rekening

```
32 case 1:  
33  
34     System.out.print("Masukkan No Rekening: ");  
35     String no = input.nextLine();  
36  
37     System.out.print("Masukkan Nama Pemilik: ");  
38     String nama = input.nextLine();  
39  
40     System.out.print("Masukkan Saldo Awal: ");  
41     double saldo = input.nextDouble();
```

Case 1 meminta tiga data awal, yaitu nomor rekening, nama pemilik, dan saldo. `nextLine` digunakan untuk membaca data berbentuk teks, sedangkan `nextDouble` digunakan untuk membaca saldo sebagai angka. Data tersebut kemudian dipakai untuk membuat object Rekening.

h. Membuat dan menyimpan rekening

```
Rekening rekeningBaru = new Rekening(no, nama, saldo);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

Object Rekening dibuat menggunakan constructor dengan data yang telah dimasukkan. Setelah dibuat, object ditambahkan ke `daftarRekening` menggunakan `add`. Object yang sama kemudian dijadikan `akunAktif` sehingga transaksi berikutnya menggunakan rekening tersebut.

i. Case 2 setor tunai

```
49 case 2:
50     if (akunAktif == null) {
51         System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
52     } else {
53         System.out.print("Masukkan nominal setor: ");
54         double setor = input.nextDouble();
55         akunAktif.setorTunai(setor);
56     }
57     break;
```

Case 2 digunakan untuk setor tunai. Program lebih dulu memastikan akun aktif tidak null. Jika tersedia, nominal dibaca menggunakan `nextDouble` dan method `setorTunai` dipanggil pada akun aktif. Setelah case selesai, `break` mengembalikan program ke menu.

j. Case 3 tarik tunai

```
61 case 3:
62     if (akunAktif == null) {
63         System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
64     } else {
65         System.out.print("Masukkan nominal tarik: ");
66         double tarik = input.nextDouble();
67         akunAktif.tarikTunai(tarik);
68     }
69     break;
```

Case 3 menjalankan proses tarik tunai. Pemeriksaan akun aktif dilakukan agar method tidak dipanggil pada object yang belum ada. Setelah nominal dibaca, method tarikTunai menjalankan validasi saldo dan minimum penarikan.

k. Case 4 cek informasi

```
72
73 case 4:
74     if (akunAktif == null) {
75         System.out.println("Error: Anda belum membuka rekening!");
76     } else {
77         akunAktif.cekInformasi();
78     }
79     break;
```

Case 4 digunakan untuk menampilkan informasi rekening. Jika belum ada akun aktif, program menampilkan pesan kesalahan. Jika ada, method cekInformasi dipanggil untuk menampilkan nomor rekening, nama pemilik, dan saldo.

l. Case 5 ganti akun

```
80
81 case 5:
82     System.out.print("Masukkan nomor rekening: ");
83     String nomorCari = input.nextLine();
84     boolean ditemukan = false;
```

Case 5 meminta nomor rekening yang akan dicari. Variabel ditemukan diberi nilai false sebagai kondisi awal sebelum pencarian dilakukan. Selanjutnya program memeriksa semua rekening yang tersimpan.

m. Mencari rekening

```
85     for (Rekening rekening : daftarRekening) {
86
87         if (rekening.nomorRekening.equals(nomorCari)) {
88
89             akunAktif = rekening;
90             ditemukan = true;
91             System.out.println("Akun berhasil diganti.");
92
93             break;
94         }
95     }
```

For-each memeriksa object Rekening satu per satu. equals digunakan untuk membandingkan nomor rekening yang tersimpan dengan input pengguna. Jika

cocok, akunAktif diganti, ditemukan menjadi true, pesan keberhasilan ditampilkan, lalu break menghentikan pencarian.

n. Menangani rekening tidak ditemukan

```
if (!ditemukan) {  
    System.out.println("Rekening tidak ditemukan.");  
}  
break;
```

Tanda ! membalik nilai boolean. Jika ditemukan masih false, berarti nomor rekening tidak ditemukan dan program menampilkan pesan. Setelah proses case 5 selesai, break mengembalikan program ke menu.

o. Case 6 cetak mutase

```
102 case 6:  
103     if (akunAktif == null) {  
104         System.out.println("Error: Anda belum membuka rekening!");  
105     } else {  
106         akunAktif.cetakMutasi();  
107     }  
108     break;
```

Case 6 menghubungkan menu utama dengan fitur cetakMutasi. Program memeriksa akun aktif terlebih dahulu, kemudian memanggil method cetakMutasi jika rekening tersedia. Dengan cara ini pengguna dapat melihat seluruh riwayat transaksi rekening aktif.

p. Case 0 keluar

```
110 case 0:  
111     isRunning = false;  
112     System.out.println("Sistem ditutup. Terima kasih!");  
113     break;
```

Case 0 digunakan untuk menghentikan program. Nilai isRunning diubah menjadi false sehingga kondisi while tidak lagi terpenuhi. Pesan penutup kemudian ditampilkan sebelum program keluar dari perulangan.

q. Default dan penutupan input

```

115 default:
116     System.out.println("Pilihan tidak valid!");
117 }
118 }
119
120 input.close();
121 }
122 }

```

Default dijalankan apabila pilihan pengguna tidak sesuai dengan case yang tersedia. Setelah perulangan selesai, `input.close` digunakan untuk menutup Scanner. Penutupan tersebut menandai bahwa program sudah tidak menggunakan sumber input lagi.

C. HASIL PENGUJIAN

2.9 Hasil Pengujian Praktikum

Bisa. Kalau bagian **2.9 Hasil Pengujian Praktikum** mau dibuat **dari awal sampai akhir berdasarkan semua screenshot yang kamu kirim**, susunannya seperti ini. Ini sudah memasukkan pengujian rekening kosong yang terakhir.

2.9 Hasil Pengujian Praktikum

a. Pengujian 1 – Membuka Rekening Baru

```
Console X
Main (2) [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse.justj.openjdk.hot
==SISTEM PERBANKAN MINI==

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 1
Masukkan No Rekening: 230806
Masukkan Nama Pemilik: lexi
Masukkan Saldo Awal: 3000000
Rekening atas nama lexi berhasil dibuat dengan saldo Rp3000000.0
```

Pada pengujian pertama, dipilih menu **1. Buka Rekening Baru**. Data yang dimasukkan adalah nomor rekening 230806, nama pemilik lexi, dan saldo awal sebesar Rp3.000.000. Setelah seluruh data dimasukkan, program menampilkan pesan bahwa rekening atas nama lexi berhasil dibuat dengan saldo awal tersebut. Hasil ini menunjukkan bahwa proses pembuatan objek Rekening berhasil dilakukan dan akun dapat digunakan untuk transaksi berikutnya.

b. Pengujian 2 – Setor Tunai

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 2
Masukkan nominal setor: 1000000
Setor tunai Rp1000000.0 berhasil. Saldo saat ini: Rp4000000.0
```

Setelah rekening berhasil dibuat, pengujian dilanjutkan dengan memilih menu **2. Setor Tunai**. Nominal yang dimasukkan adalah Rp1.000.000. Program menampilkan bahwa setor tunai berhasil dan saldo berubah dari Rp3.000.000 menjadi Rp4.000.000. Transaksi penyetoran tersebut juga dicatat ke dalam riwayatTransaksi sebagai transaksi **Kredit**. Pencatatan transaksi setor merupakan

bagian dari pengembangan pada Praktikum 2, yaitu membuat objek Transaksi ketika aktivitas penyetoran berhasil.

c. Pengujian 3 – Tarik Tunai Berhasil

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan nominal tarik: 500000
Tarik tunai Rp500000.0 berhasil. Saldo saat ini: Rp3500000.0
```

Selanjutnya dipilih menu **3. Tarik Tunai** dengan nominal penarikan sebesar Rp500.000. Karena saldo rekening mencukupi, transaksi penarikan berhasil dilakukan. Saldo rekening kemudian berubah dari Rp4.000.000 menjadi Rp3.500.000. Transaksi penarikan tersebut dicatat sebagai **Debit** dengan ID yang menggunakan prefiks TRX-T-. Pengujian ini membuktikan bahwa **Tugas 1**, yaitu integrasi rekam jejak penarikan, telah diterapkan pada program.

d. Pengujian 4 – Cek Informasi Rekening

```
Main (2) [Java Application] C:\Users\Hype AMD'
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 230806
Nama Pemilik : lexi
Saldo Akhir  : Rp3500000.0
-----
```

Setelah melakukan setor dan tarik tunai, dipilih menu **4. Cek Informasi Rekening**. Program menampilkan nomor rekening 230806, nama pemilik lexi, dan saldo akhir sebesar Rp3.500.000. Hasil tersebut menunjukkan bahwa perubahan saldo dari transaksi sebelumnya telah tersimpan dengan benar pada objek rekening.

e. Pengujian 5 – Ganti Akun

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 5
Masukkan nomor rekening: 230806
Akun berhasil diganti.
```

Berikutnya dilakukan pengujian menu **5. Ganti Akun**. Nomor rekening 230806 dimasukkan sebagai nomor rekening yang ingin digunakan. Program kemudian menampilkan pesan “**Akun berhasil diganti.**” Pengujian ini memastikan bahwa program dapat memilih kembali rekening yang tersimpan dalam daftarRekening sebagai akun aktif.

f. Pengujian 6 – Cetak Mutasi Rekening

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 6
==MUTASI REKENING==
ID Transaksi : TRX-S-1 | Jenis : Kredit | Nominal : Rp1000000.0
ID Transaksi : TRX-T-2 | Jenis : Debit | Nominal : Rp500000.0
```

Setelah transaksi setor dan tarik berhasil dilakukan, dipilih menu **6. Cetak Mutasi (Riwayat)**. Program menampilkan seluruh transaksi yang tersimpan dalam riwayat rekening. Hasil yang ditampilkan adalah transaksi TRX-S-1 dengan jenis **Kredit** sebesar Rp1.000.000 dan transaksi TRX-T-2 dengan jenis **Debit** sebesar Rp500.000.

g. Pengujian 7 – Tarik Tunai Gagal

```
Console X
Main (2) [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse.justj.openjd

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan nominal tarik: 4000000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp3500000.0
```

Untuk menguji kondisi penarikan yang tidak berhasil, dilakukan penarikan sebesar Rp4.000.000, sedangkan saldo rekening yang tersedia hanya Rp3.500.000. Program menolak transaksi tersebut dan menampilkan pesan **“Transaksi Gagal: Saldo tidak mencukupi.”** Saldo rekening tetap sebesar Rp3.500.000. Pengujian ini penting karena pada modul dijelaskan bahwa objek transaksi penarikan hanya boleh dicatat apabila penarikan berhasil atau saldo mencukupi.

h. Pengujian 8 – Memastikan Penarikan Gagal Tidak Masuk Mutasi

```
Console X
Main (2) [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse.justj.openjdk.f

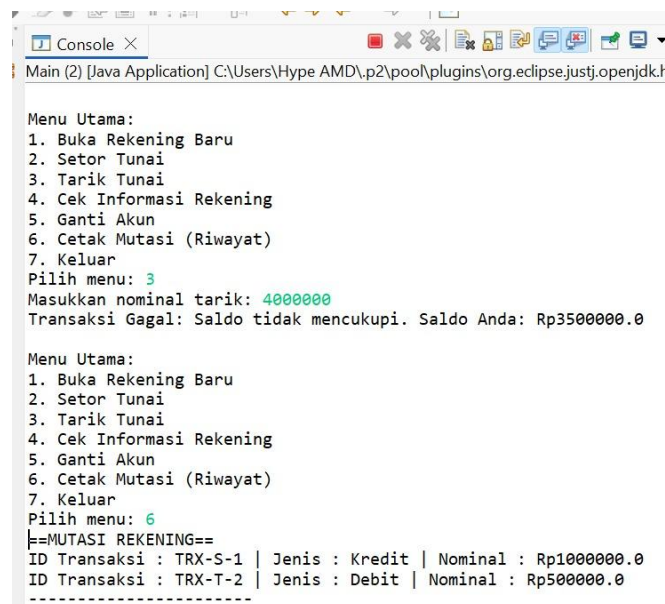
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan nominal tarik: 4000000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp3500000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 6
==MUTASI REKENING==
ID Transaksi : TRX-S-1 | Jenis : Kredit | Nominal : Rp1000000.0
ID Transaksi : TRX-T-2 | Jenis : Debit | Nominal : Rp500000.0
-----
```

Setelah penarikan sebesar Rp4.000.000 gagal, menu **6. Cetak Mutasi (Riwayat)** dipilih kembali. Hasil yang ditampilkan tetap hanya dua transaksi sebelumnya, yaitu TRX-S-1 sebagai **Kredit** dan TRX-T-2 sebagai **Debit**. Tidak muncul transaksi baru sebesar Rp4.000.000.

Hasil tersebut menunjukkan bahwa program tidak mencatat penarikan yang gagal ke dalam riwayatTransaksi. Dengan demikian, pencatatan transaksi Debit hanya dilakukan setelah proses penarikan berhasil, sesuai dengan ketentuan **Tugas 1** pada modul.

- i. **Pengujian 9 – Cetak Mutasi pada Rekening Tanpa Transaksi**Sebagai pengujian terakhir, dibuat rekening baru dengan nomor rekening 12345, nama pemilik nana, dan saldo awal sebesar Rp1.000.000. Setelah rekening berhasil dibuat, tidak dilakukan setor maupun tarik tunai. Pengguna langsung memilih menu **6. Cetak Mutasi (Riwayat)**.



```
Console X
Main (2) [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse.justj.openjdk.f

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan nominal tarik: 4000000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp3500000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 6
|=MUTASI REKENING==
ID Transaksi : TRX-S-1 | Jenis : Kredit | Nominal : Rp1000000.0
ID Transaksi : TRX-T-2 | Jenis : Debit | Nominal : Rp500000.0
-----
```

Hasil tersebut menunjukkan bahwa program dapat menangani kondisi ketika riwayatTransaksi masih kosong. Pengujian ini sesuai dengan instruksi **Tugas 2** pada modul yang meminta program menampilkan pesan tersebut apabila belum terdapat transaksi pada rekening.

j. Kesimpulan Hasil Pengujian

Berdasarkan seluruh pengujian yang telah dilakukan, program dapat menjalankan proses pembuatan rekening, setor tunai, tarik tunai, pengecekan informasi rekening, pergantian akun, dan pencetakan mutasi. Selain itu, **Tugas 1** berhasil diuji dengan menunjukkan bahwa penarikan yang berhasil dicatat sebagai **Debit** menggunakan prefiks TRX-T-, sedangkan penarikan yang gagal tidak dicatat. **Tugas 2** juga berhasil diuji karena program dapat menampilkan seluruh riwayat transaksi dan memberikan pesan **“Belum ada transaksi pada rekening ini”** ketika riwayat masih kosong. Dengan demikian, hasil pengujian sudah mencakup bagian-bagian utama yang diperintahkan dalam modul Praktikum 2.

BAB III

PENUTUP

3.1 Kesimpulan

Praktikum 2 memberikan pemahaman tentang penerapan relasi antar objek pada Java melalui pengembangan program perbankan. Class Rekening memiliki hubungan dengan banyak object Transaksi yang disimpan menggunakan ArrayList. Setiap transaksi yang berhasil dapat direkam dan ditampilkan kembali melalui fitur mutasi rekening.

Melalui latihan dan tugas, konsep constructor, object, ArrayList, percabangan, perulangan for-each, method, dan switch-case dapat diterapkan dalam satu program. Tugas 1 menambahkan rekam jejak penarikan, sedangkan Tugas 2 menambahkan fitur cetak mutasi dan menu baru pada Main.

3.2 Saran

Untuk praktikum berikutnya, program dapat dikembangkan dengan menambahkan tanggal transaksi, pencarian transaksi berdasarkan ID, filter berdasarkan jenis transaksi, serta penyimpanan data secara permanen. Pengembangan tersebut dapat membantu memperdalam pemahaman mengenai hubungan antar class dan pengelolaan object dalam Java.

DAFTAR PUSTAKA

Modul Praktikum Pemrograman Berorientasi Objek. Materi Praktikum 2: Relasi Antar Objek (Class Diagram & Asosiasi).

Oracle. Java Documentation. Dokumentasi Java mengenai class, object, ArrayList, Scanner, dan struktur kontrol program.

